Actividad evaluativa 1
Centro de monitoreo de accesos

Asier Gutierrez Hinojo

23 de octubre de 2026

Índice

1. Atributos de la clase Servidor **3**
1.1. Nombre del servidor . . . 3
1.2. Origen del acceso . . . 3
1.3. Criticidad del servidor . . . 3
1.4. Intentos totales . . . 4
1.5. Intentos fallidos . . . 4
1.6. Método ***datosValidos()*** . . . 4
1.6.1. Validación del nombre . . . 5
1.6.2. Validación del origen . . . 5
1.6.3. Validación de la criticidad . . . 5
1.6.4. Validación de los intentos . . . 5
1.7. Método ***evaluarEstado()*** . . . 5
1.7.1. Estado DATOS INVALIDOS . . . 6
1.7.2. Estado BLOQUEADO . . . 6
1.7.3. Estado ALERTA . . . 6
1.7.4. Estado NORMAL . . . 7
1.8. Método ***porcentajeFallos()*** . . . 7
1.9. Método ***mostrarResumen()*** . . . 7

2. Métodos de la clase Main **8**
2.1. Método ***leerServidor()*** . . . 8
2.2. Método ***nivelPrioridad()*** . . . 8
2.3. Método ***compararServidores()*** . . . 9
2.4. Método ***main()*** . . . 10
2.4.1. Creación del Scanner . . . 10
2.4.2. Creación de los dos objetos . . . 10
2.4.3. Lectura de los datos . . . 10
2.4.4. Presentación de los resúmenes . . . 11
2.4.5. Comparación de prioridades . . . 11
2.4.6. Cierre del Scanner . . . 11

3. Pruebas y resultados **11**
3.1. Casos de prueba obligatorios . . . 11
3.2. Comparación entre servidores . . . 11
3.3. Ejemplo de ejecución . . . 12

4. Preguntas **12**
4.1. ¿Por qué dos objetos de la misma clase pueden almacenar valores diferentes? . . . 12
4.2. ¿Qué diferencia existe entre Servidor b = a; y Servidor b = new Servidor();? . . . 12
4.3. ¿Por qué es importante validar antes de calcular el porcentaje? 13

5. Repositorio en GitHub **13**

6. Código completo **13**

1. Atributos de la clase Servidor

La clase `Servidor` almacena la información necesaria para analizar los intentos de autenticación de cada servidor. Sus atributos son de instancia, haciendo que cada objeto mantenga sus propios valores de forma independiente.

Por otro lado, en este apartado tambien se explican los métodos que permiten validar los datos, clasificar el riesgo y mostrar los resultados.

1.1. Nombre del servidor

El atributo `nombre` es de tipo `String` y almacena el nombre que identifica al servidor.

```
String nombre;
```

Su valor se introduce por teclado utilizando el método ***nextLine()***, que permite leer la línea completa, incluyendo los espacios dentro de el:

```
servidor.nombre = teclado.nextLine();
```

El método ***datosValidos()*** comprueba que el nombre no esté vacío ni formado únicamente por espacios:

```
!nombre.trim().isEmpty()
```

El método ***trim()*** elimina los espacios exteriores y el método ***isEmpty()*** comprueba si el resultado está vacío. El operador `!` invierte el resultado para asegurar que exista algún carácter.

1.2. Origen del acceso

El atributo `origen` es de tipo `String` e indica si los accesos proceden de un origen interno o externo.

```
String origen;
```

El método ***datosValidos()*** comprueba que su valor sea una de las dos opciones permitidas:

```
(origen.trim().equalsIgnoreCase("interno") || origen.trim().
    equalsIgnoreCase("externo"))
```

El operador `||` significa .ºo", haciendo que solo baste con una comparación verdadera. El método ***equalsIgnoreCase()*** ignora las diferencias entre mayúsculas y minúsculas, mientras que ***trim()*** elimina los espacios exteriores.

Este atributo interviene en la clasificación del riesgo. Por ejemplo, un servidor externo con dos fallos se clasifica como `ALERTA`, siempre que los datos sean válidos.

1.3. Criticidad del servidor

El atributo `critico` es de tipo `String` e indica si el servidor se considera crítico.

```
String critico;
```

Solo se permiten los valores `si` y `no`. El método ***datosValidos()*** comprueba estas opciones ignorando mayúsculas y espacios exteriores:

```
(critico.trim().equalsIgnoreCase("si") || critico.trim().
    equalsIgnoreCase("no"))
```

En el método ***evaluarEstado()***, esta información se guarda en una variable de tipo `boolean`:

```
boolean esCritico = critico.trim().equalsIgnoreCase("si");
```

La variable `esCritico` será `true` si el servidor es crítico. Esto permite bloquear bloquear el servidor en caso de que su criticidad sea positiva y al menos tenga 3 fallos.

1.4. Intentos totales

El atributo `intentosTotales` es de tipo `int` y almacena el número total de intentos de autenticación registrados.

```
int intentosTotales;
```

Su valor se lee por teclado mediante el método ***nextInt()***:

```
servidor.intentosTotales = teclado.nextInt();
```

El método ***datosValidos()*** exige que este número sea mayor que cero:

```
intentosTotales > 0
```

Este atributo también se utiliza para poder calcular el porcentaje de fallos. Comprobar que sea positivo permite evitar una división entre cero y rechazar cantidades negativas.

1.5. Intentos fallidos

El atributo `intentosFallidos` es de tipo `int` y almacena el número de intentos de autenticación que han fallado.

```
int intentosFallidos;
```

Su valor se introduce mediante el método ***nextInt()***:

```
servidor.intentosFallidos = teclado.nextInt();
```

El método ***datosValidos()*** comprueba que no sea negativo ni supere el total de intentos:

```
intentosFallidos >= 0 && intentosFallidos <= intentosTotales
```

El operador `&&` exige que ambas condiciones se cumplan. Por ejemplo, tres fallos de diez intentos son válidos, pero cinco fallos de cuatro intentos no lo son.

Este atributo se utiliza en el método ***evaluarEstado()*** para clasificar el riesgo y en el método ***porcentajeFallos()*** para calcular el porcentaje de fallos.

1.6. Método *datosValidos()*

El método ***datosValidos()*** comprueba que todos los atributos del servidor cumplan las condiciones del enunciado. Devuelve un valor de tipo `boolean`: `true` si todos los datos son válidos y `false` si alguna condición falla.

```
boolean datosValidos() {
    return !nombre.trim().isEmpty()
            && (origen.trim().equalsIgnoreCase("interno")
                || origen.trim().equalsIgnoreCase("externo"))
            && (critico.trim().equalsIgnoreCase("si")
                || critico.trim().equalsIgnoreCase("no"))
            && intentosTotales > 0
            && intentosFallidos >= 0
            && intentosFallidos <= intentosTotales;
}
```

El operador `&&` une las comprobaciones y exige que todas sean verdaderas. La instrucción `return` devuelve el resultado de esta expresión. Este método comprueba si los datos son validos para poder clasificarlos posteriormente.

1.6.1. Validación del nombre

La primera condición comprueba que el nombre contenga al menos un carácter distinto de un espacio:

```
1 !nombre.trim().isEmpty()
```

El método ***trim()*** elimina los espacios del principio y del final. Después, el método ***isEmpty()*** devuelve `true` si la cadena resultante está vacía.

Como se necesita que el nombre no esté vacío, se utiliza el operador de negación `!`, es decir niega el resultado obtenido asegurando que contenga al menos un carácter. Por ejemplo, `"VPN "` es válido, mientras que no lo es.

1.6.2. Validación del origen

Esta condición comprueba que el origen sea `interno` o `externo`:

```
1 (origen.trim().equalsIgnoreCase("interno") || origen.trim().
    equalsIgnoreCase("externo"))
```

El operador `||` permite que se cumpla cualquiera de las dos comparaciones. Los paréntesis agrupan ambas opciones para tratarlas como una única condición dentro de la validación general.

El método ***trim()*** elimina los espacios exteriores y el método ***equalsIgnoreCase()*** compara los textos sin distinguir mayúsculas y minúsculas. Por ejemplo, si se introduce `.ᴱˣTERNO ` es válido, pero `"local"` no lo es.

1.6.3. Validación de la criticidad

La criticidad solo puede contener los valores `si` o `no`. Esta expresión se encarga de comprobar ambas posibilidades:

```
1 (critico.trim().equalsIgnoreCase("si") || critico.trim().
    equalsIgnoreCase("no"))
```

El operador `||` permite aceptar cualquiera de los dos valores. Al utilizar los métodos ***trim()*** y ***equalsIgnoreCase()***, aseguramos que se admitan entradas como `"SI "` o `"No"`.

Un valor diferente, como `"quizas"`, hace que esta condición sea falsa y que los datos del servidor se consideren inválidos.

1.6.4. Validación de los intentos

Los contadores deben cumplir tres condiciones: el total debe ser positivo, los fallos no pueden ser negativos y los fallos no pueden superar el total.

```
1 intentosTotales > 0 && intentosFallidos >= 0 && intentosFallidos <=
    intentosTotales
```

La primera condición asegura que el total no sea cero o negativo. La segunda permite cero fallos, pero rechaza cantidades negativas. La tercera evita situaciones que no son posibles, como por ejemplo, registrar cinco fallos cuando solo se han realizado cuatro intentos.

Las tres condiciones están unidas con `&&`, asegurándose que las tres tengan que cumplirse.

1.7. Método *evaluarEstado()*

El método ***evaluarEstado()*** analiza los datos del servidor y devuelve su estado generando un `String`. Los posibles resultados son `DATOS INVALIDOS`, `BLOQUEADO`, `ALERTA` y `NORMAL`.

Primero comprueba si los datos son validos. Si lo son, obtiene dos variables booleanas que agilizan la lectura de las reglas:

```
boolean externo = origen.trim().equalsIgnoreCase("externo");
boolean esCritico = critico.trim().equalsIgnoreCase("si");
```

La variable `externo` indica si el origen es externo y `esCritico` indica si el servidor es crítico.

Las condiciones se comprueban en el orden establecido: datos inválidos, bloqueo, alerta y normalidad. Cada instrucción `return` termina el método, evitando que una condición posterior sustituya un estado de mayor prioridad.

1.7.1. Estado DATOS INVALIDOS

Para determinar si los datos son inválidos se llama al método ***datosValidos()***:

```
if (!datosValidos()) {
    return "DATOS INVALIDOS";
}
```

Si la validación devuelve `false`, el operador `!` convierte el resultado en `true` y se ejecuta el bloque del `if`.

La instrucción `return` devuelve `"DATOS INVALIDOS"` y termina el método. De esta forma, no se clasifica el riesgo utilizando información incorrecta.

1.7.2. Estado BLOQUEADO

Con datos válidos, se comprueba primero si corresponde bloquear el servidor:

```
if (intentosFallidos >= 5 || (esCritico && externo && intentosFallidos
    >= 3)) {
    return "BLOQUEADO";
}
```

El servidor queda bloqueado cuando se cumple cualquiera de estas dos situaciones:

- Tiene cinco o más intentos fallidos.
- Es crítico, el origen es externo y tiene al menos tres intentos fallidos.

El operador `||` separa las dos posibilidades. Dentro de la segunda, los operadores `&&` exigen que las tres condiciones se cumplan a la vez.

Esta comprobación aparece antes que la alerta porque el bloqueo tiene mayor prioridad.

1.7.3. Estado ALERTA

Si el servidor no ha sido bloqueado, se comprueba si debe clasificarse en alerta:

```
if ((intentosFallidos >= 3 && intentosFallidos <= 4) || (externo &&
    intentosFallidos >= 2)) {
    return "ALERTA";
}
```

Se devuelve `."ALERTA"` cuando hay tres o cuatro fallos, o cuando el origen es externo y se han registrado al menos dos fallos, siempre que no se haya aplicado previamente el bloqueo.

Por ejemplo, un servidor interno con cuatro fallos queda en alerta. Un servidor crítico externo con tres fallos queda bloqueado, porque la condición de bloqueo se comprueba antes y termina el método.

1.7.4. Estado NORMAL

Si los datos son válidos y no se cumple ninguna condición de bloqueo ni de alerta, el método devuelve el estado normal:

```
return "NORMAL";
```

Por ejemplo, un servidor interno con dos fallos se clasifica como `NORMAL`, cuando el total de intentos es válido y no es inferior al número de fallos.

1.8. Método *porcentajeFallos()*

El método ***porcentajeFallos()*** calcula qué porcentaje del total de intentos corresponde a autenticaciones fallidas. Devuelve un valor de tipo `double`, que permite representar decimales.

```
double porcentajeFallos() {
    return intentosFallidos * 100.0 / intentosTotales;
}
```

El cálculo multiplica los intentos fallidos por cien y divide el resultado entre los intentos totales. Por ejemplo, tres fallos de diez intentos representan un 30 % de fallos.

Se utiliza `100.0` para que la operación se realice con decimales y no se pierda precisión en el resultado.

Este método no tiene en cuenta si los datos no son validos, solo presupone que su resultado es valido. El método ***mostrarResumen()*** solo lo llama cuando el método ***datosValidos()*** devuelve `true`. Si los datos son inválidos, se muestra `N/D` sin calcular el porcentaje.

1.9. Método *mostrarResumen()*

El método ***mostrarResumen()*** imprime por pantalla el nombre del servidor, los intentos fallidos respecto al total, el porcentaje de fallos y el estado de seguridad.

Su tipo de retorno es `void`, porque muestra información pero no devuelve un valor.

```
void mostrarResumen() {
    System.out.print(nombre.trim() + " | " + intentosFallidos + "/" +
        intentosTotales + " | ");

    if (datosValidos()) {
        System.out.printf(java.util.Locale.US, "%.1f%%",
            porcentajeFallos());
    } else {
        System.out.print("N/D");
    }

    System.out.println(" | " + evaluarEstado());
}
```

El método ***print()*** se utiliza para mostrar los datos iniciales sin cambiar de línea. El operador `+` permite unir los textos con los valores de los atributos.

Después se comprueba la validez de los datos. Si son válidos, se calcula el porcentaje y se muestra utilizando el método ***printf()***. El formato `%.1f` muestra un decimal y `%%` imprime el símbolo de porcentaje. El valor `java.util.Locale.US` permite utilizar el punto como separador decimal.

Si los datos son inválidos, se muestra `N/D` y no se llama al método ***porcentajeFallos()***.

Por ultimo, el método ***println()*** añade el estado que se ha obtenido en el método ***evaluarEstado()*** y termina la línea.

2. Métodos de la clase Main

La clase `Main` se encarga de coordinar la ejecución del programa: solicita los datos, crea los dos servidores, muestra sus resúmenes y compara sus prioridades.

Los métodos se declaran `static` para poder utilizarlos directamente desde el método ***main()*** sin crear un objeto de la clase `Main`.

2.1. Método *leerServidor()*

El método ***leerServidor()*** solicita los cinco datos de un servidor y los guarda en sus atributos. Recibe dos parámetros: el `Scanner` que lee la entrada y el objeto `Servidor` que debe rellenar.

```
static void leerServidor(Scanner teclado, Servidor servidor) {
    System.out.print("Nombre: ");
    servidor.nombre = teclado.nextLine();

    System.out.print("Origen (interno/externo): ");
    servidor.origen = teclado.nextLine();

    System.out.print("Critico (si/no): ");
    servidor.critico = teclado.nextLine();

    System.out.print("Intentos totales: ");
    servidor.intentosTotales = teclado.nextInt();

    System.out.print("Intentos fallidos: ");
    servidor.intentosFallidos = teclado.nextInt();

    teclado.nextLine();
}
```

Los textos se leen mediante el método ***nextLine()*** y los números enteros mediante ***nextInt()***.

La última llamada a ***nextLine()*** se asegura que la próxima linea que se lea no sea una cadena de texto. Sin ella, el nombre del siguiente servidor podría leerse como una cadena vacía.

El método es `void` porque guarda los datos directamente en el objeto y no necesita devolver un resultado.

2.2. Método *nivelPrioridad()*

El método ***nivelPrioridad()*** recibe un servidor y transforma su estado en un número entero para facilitar la comparación.

```
static int nivelPrioridad(Servidor servidor) {
    String estado = servidor.evaluarEstado();

    if (estado.equals("BLOQUEADO")) {
        return 2;
    }

    if (estado.equals("ALERTA")) {
        return 1;
    }

    if (estado.equals("NORMAL")) {
        return 0;
    }

    return -1;
```

```
}
```

Primero se obtiene el estado mediante el método ***evaluarEstado()***. Después, el método ***equals()*** compara ese resultado con cada estado válido.

Los niveles quedan como:

- `BLOQUEADO`: nivel 2.
- `ALERTA`: nivel 1.
- `NORMAL`: nivel 0.
- `DATOS INVALIDOS`: valor -1.

El valor -1 indica que no se puede realizar la comparación. Para los estados válidos, un número mayor representa una prioridad superior.

2.3. Método *compararServidores()*

El método ***compararServidores()*** recibe los dos servidores y muestra cuál tiene atención prioritaria.

```
static void compararServidores(Servidor primero, Servidor segundo) {
    int nivelPrimero = nivelPrioridad(primero);
    int nivelSegundo = nivelPrioridad(segundo);

    if (nivelPrimero == -1 || nivelSegundo == -1) {
        System.out.println("COMPARACION NO DISPONIBLE");
    } else if (nivelPrimero == nivelSegundo) {
        System.out.println("IGUAL PRIORIDAD");
    } else if (nivelPrimero > nivelSegundo) {
        System.out.println("ATENCION PRIORITARIA: " + primero.nombre.
            trim());
    } else {
        System.out.println("ATENCION PRIORITARIA: " + segundo.nombre.
            trim());
    }
}
```

Primero obtiene los niveles mediante el método ***nivelPrioridad()***. Después aplica las condiciones en este orden:

1. Si algún servidor tiene datos inválidos, muestra `COMPARACION NO DISPONIBLE`.
2. Si ambos tienen el mismo nivel válido, muestra `IGUAL PRIORIDAD`.
3. Si el primer servidor tiene mayor nivel, muestra su nombre.
4. En caso contrario, el segundo servidor tiene mayor nivel y se muestra su nombre.

La comparación depende de los estados que se han calculado.

El método se declara `void` porque imprime el resultado y no devuelve ningún valor.

2.4. Método *main()*

El método ***main()*** es el punto de entrada del programa. Desde este método se ejecutan las operaciones necesarias para analizar los dos servidores.

```
public static void main(String[] args) {
    Scanner teclado = new Scanner(System.in);

    Servidor primero = new Servidor();
    Servidor segundo = new Servidor();

    System.out.println("DATOS DEL PRIMER SERVIDOR");
    leerServidor(teclado, primero);

    System.out.println("DATOS DEL SEGUNDO SERVIDOR");
    leerServidor(teclado, segundo);

    System.out.println("========== REPORTE DE SEGURIDAD ==========");
    primero.mostrarResumen();
    segundo.mostrarResumen();

    compararServidores(primero, segundo);
    teclado.close();
}
```

La palabra `public` permite que Java acceda al método para iniciar la ejecución. La palabra `static` permite ejecutarlo sin crear un objeto de la clase `Main` y `void` indica que no devuelve ningún valor.

2.4.1. Creación del Scanner

Se crea un objeto `Scanner`:

```
Scanner teclado = new Scanner(System.in);
```

La variable `teclado` permite acceder a los métodos de lectura. El valor `System.in` indica que los datos se reciben desde la entrada estándar.

Se utiliza el mismo lector para introducir los datos de los dos servidores.

2.4.2. Creación de los dos objetos

Se crean dos objetos independientes de la clase `Servidor`:

```
Servidor primero = new Servidor();
Servidor segundo = new Servidor();
```

Cada `new Servidor()` crea un objeto diferente. Haciendo que los atributos de `primero` y `segundo` se almacenan de manera independiente.

2.4.3. Lectura de los datos

Se llama al método ***leerServidor()*** una vez para cada objeto:

```
System.out.println("DATOS DEL PRIMER SERVIDOR");
leerServidor(teclado, primero);

System.out.println("DATOS DEL SEGUNDO SERVIDOR");
leerServidor(teclado, segundo);
```

Los mensajes permiten identificar a qué servidor corresponden los datos que se están introduciendo.

2.4.4. Presentación de los resúmenes

Después de leer los datos, se llama al método ***mostrarResumen()*** de cada servidor:

```
System.out.println("========== REPORTE DE SEGURIDAD ==========");
primero.mostrarResumen();
segundo.mostrarResumen();
```

2.4.5. Comparación de prioridades

Una vez mostrados los resúmenes, se comparan los dos servidores:

```
compararServidores(primero, segundo);
```

El método ***compararServidores()*** obtiene los niveles de prioridad.

2.4.6. Cierre del Scanner

Al terminar la lectura y el análisis, se cierra el `Scanner`:

```
teclado.close();
```

El método ***close()*** cierra el lector y su entrada asociada. Se coloca al final porque el programa ya no necesita recibir más datos.

3. Pruebas y resultados

Las pruebas permiten comprobar que el programa valida correctamente los datos, respeta el orden de las reglas de seguridad y compara los servidores según su estado.

3.1. Casos de prueba obligatorios

Cada fila representa una prueba independiente de un objeto `Servidor`.

Nombre	Origen	Crítico	Total	Fallos	Estado esperado
VPN	externo	si	10	3	BLOQUEADO
WEB	interno	no	8	4	ALERTA
CORREO	externo	no	8	2	ALERTA
VPN	interno	si	12	2	NORMAL
WEB	externo	no	5	5	BLOQUEADO
WEB	interno	no	4	5	DATOS INVALIDOS
WEB	interno	no	0	0	DATOS INVALIDOS
CORREO	local	no	10	2	DATOS INVALIDOS
WEB	externo	quizás	10	2	DATOS INVALIDOS

3.2. Comparación entre servidores

La comparación debe respetar la prioridad: `BLOQUEADO` tiene mayor prioridad que `ALERTA`, y `ALERTA` tiene mayor prioridad que `NORMAL`.

Primer servidor	Segundo servidor	Resultado esperado
BLOQUEADO	ALERTA	Prioridad del primero
ALERTA	BLOQUEADO	Prioridad del segundo
ALERTA	NORMAL	Prioridad del primero
NORMAL	ALERTA	Prioridad del segundo
BLOQUEADO	NORMAL	Prioridad del primero
NORMAL	BLOQUEADO	Prioridad del segundo
NORMAL	NORMAL	IGUAL PRIORIDAD
ALERTA	ALERTA	IGUAL PRIORIDAD
BLOQUEADO	BLOQUEADO	IGUAL PRIORIDAD

Si alguno de los servidores tiene `DATOS INVALIDOS`, el resultado debe ser `COMPARACION NO DISPONIBLE`.

3.3. Ejemplo de ejecución

Se introducen los siguientes datos:

- Primer servidor: nombre `VPN`, origen `EXTERNO`, criticidad `SI`, diez intentos totales y tres fallidos.
- Segundo servidor: nombre `CORREO`, origen `interno`, criticidad `no`, doce intentos totales y tres fallidos.

El reporte esperado al introducir estos datos es el siguiente:

```
========== REPORTE DE SEGURIDAD ==========
VPN | 3/10 | 30.0% | BLOQUEADO
CORREO | 3/12 | 25.0% | ALERTA
ATENCION PRIORITARIA: VPN
```

El servidor VPN queda bloqueado porque es crítico, tiene origen externo y acumula tres fallos. El servidor CORREO queda en alerta porque tiene tres fallos y no cumple ninguna condición de bloqueo.

4. Preguntas

4.1. ¿Por qué dos objetos de la misma clase pueden almacenar valores diferentes?

Porque cada objeto tiene sus propios atributos de instancia. La clase define qué datos y métodos tendrán los objetos, pero cada objeto almacena sus propios valores.

```
Servidor primero = new Servidor();
Servidor segundo = new Servidor();

primero.nombre = "VPN";
segundo.nombre = "CORREO";
```

En este ejemplo se crean dos objetos distintos. Aunque los dos pertenecen a la misma clase, el atributo `nombre` del primero contiene `"VPN"` y el del segundo contiene `ÇORREO"`.

Esto es posible porque los atributos de `Servidor` no se han declarado `static`.

4.2. ¿Qué diferencia existe entre Servidor b = a; y Servidor b = new Servidor();?

La instrucción `Servidor b = a;` copia la información que esta almacenada en `a`. No crea otro objeto, ambas variables pertenecen al mismo servidor.

```
Servidor a = new Servidor();
a.nombre = "VPN";

Servidor b = a;
b.nombre = "WEB";

System.out.println(a.nombre); // WEB
```

En cambio, `Servidor b = new Servidor();` crea otro objeto independiente:

```java
Servidor a = new Servidor();
a.nombre = "VPN";

Servidor b = new Servidor();
b.nombre = "WEB";

System.out.println(a.nombre); // VPN
```

En este segundo caso, modificar los atributos de `b` no cambia los atributos de `a`.

4.3. ¿Por qué es importante validar antes de calcular el porcentaje?

Porque el cálculo solo tiene sentido si los datos son correctos. Un total igual a cero provocaría una división entre cero, por otro lado, los valores negativos o los fallos superiores al total generarían porcentajes imposibles, que no tendrían ningún tipo de sentido.

Por este motivo, el método ***mostrarResumen()*** comprueba primero la validez:

```java
if (datosValidos()) {
    System.out.printf(java.util.Locale.US,
            "%.1f%%", porcentajeFallos());
} else {
    System.out.print("N/D");
}
```

De esta manera el método ***porcentajeFallos()*** solo se ejecuta cuando todos los datos cumplen las reglas.

5. Repositorio en GitHub

`https://github.com/AsierGutierrez-ciber/Centro_Monitoreo_Accessos`

6. Código completo

```java
import java.util.Scanner;

class Servidor {
    String nombre; //nombre servidor
    String origen; //interno o externo
    String critico; //si o no
    int intentosTotales;
    int intentosFallidos;

    boolean datosValidos() {
        return !nombre.trim().isEmpty()
            && (origen.trim().equalsIgnoreCase("interno")
                || origen.trim().equalsIgnoreCase("externo"))
            && (critico.trim().equalsIgnoreCase("si")
                || critico.trim().equalsIgnoreCase("no"))
            && intentosTotales > 0
            && intentosFallidos >= 0
            && intentosFallidos <= intentosTotales;

    }

    String evaluarEstado() {
```

```
23          if (!datosValidos()) {
24              return "DATOS INVALIDOS";
25          }
26
27          boolean externo = origen.trim().equalsIgnoreCase("externo");
28          boolean esCritico = critico.trim().equalsIgnoreCase("si");
29
30          if (intentosFallidos >=5 || (esCritico && externo &&
                intentosFallidos>=3)) {
31              return "BLOQUEADO";
32          }
33
34          if ((intentosFallidos >=3 && intentosFallidos <=4) || (externo
                && intentosFallidos >=2)) {
35                  return "ALERTA";
36          }
37
38          return "NORMAL";
39      }
40
41      double porcentajeFallos() {
42          return intentosFallidos * 100.0 / intentosTotales;
43      }
44
45      void mostrarResumen() {
46          System.out.print(nombre.trim() + " | " + intentosFallidos + "/"
                + intentosTotales + " | ");
47
48          if (datosValidos()) {
49              System.out.printf(java.util.Locale.US, "%.1f%%",
                    porcentajeFallos());
50          } else {
51
52              System.out.print("N/D");
53          }
54
55          System.out.println(" | " + evaluarEstado());
56      }
57  }
58
59  public class Main {
60      static void leerServidor(Scanner teclado, Servidor servidor) {
61          System.out.print("Nombre: ");
62          servidor.nombre = teclado.nextLine();
63          System.out.print("Origen (Interno/Externo): ");
64          servidor.origen = teclado.nextLine();
65          System.out.print("Critico (Si/no): ");
66          servidor.critico = teclado.nextLine();
67          System.out.print("Intentos Totales: ");
68          servidor.intentosTotales = teclado.nextInt();
69          System.out.print("Intentos Fallidos: ");
70          servidor.intentosFallidos = teclado.nextInt();
71
72          teclado.nextLine();
73      }
74
75      static int nivelPrioridad(Servidor servidor) {
76          String estado = servidor.evaluarEstado();
```

```
77          if (estado.equals("BLOQUEADO")) {
78              return 2;
79          }
80
81          if (estado.equals("ALERTA")) {
82              return 1;
83          }
84
85          if (estado.equals("NORMAL")) {
86              return 0;
87          }
88
89          return -1;
90      }
91
92      static void compararServidores(Servidor primero, Servidor segundo) {
93          int nivelPrimero = nivelPrioridad(primero);
94          int nivelSegundo = nivelPrioridad(segundo);
95
96          if (nivelPrimero == -1 || nivelSegundo == -1){
97              System.out.println("COMPARACION NO DISPONIBLE");
98          } else if (nivelPrimero == nivelSegundo) {
99              System.out.println("IGUAL PRIORIDAD");
100         } else if (nivelPrimero > nivelSegundo) {
101             System.out.println("ATENCION PRIORITARIA: " + primero.nombre
                    .trim());
102         } else {
103             System.out.println("ATENCION PRIORITARIA: " + segundo.nombre
                    .trim());
104         }
105     }
106
107     public static void main(String [] args) {
108         Scanner teclado = new Scanner(System.in);
109
110         Servidor primero = new Servidor();
111         Servidor segundo = new Servidor();
112
113         System.out.println("DATOS PRIMERO SERVIDOR: ");
114         leerServidor(teclado, primero);
115         System.out.println("DATOS SEGUNDO SERVIDOR: ");
116         leerServidor(teclado, segundo);
117
118         System.out.println("========== REPORTE DE SEGURIDAD ==========")
                ;
119         primero.mostrarResumen();
120         segundo.mostrarResumen();
121         compararServidores(primero, segundo);
122         teclado.close();
123     }
124 }
```